EDUGENIE

Google Gemini Powered Learning Assistant

NAAN MUDHALVAN PROJECT DOCUMENTATION

NAAN MUDHALVAN PROJECT DOCUMENTATION

A 50-Page Project Report

Submitted by: Tella Divya Sree

Mentor: Siri

Date: 11/04/2025

Project theme: Generative AI for Personalized Learning

Technology focus: FastAPI, HTML/CSS, Jinja2, Google Gemini API, LaMini-Flan-T5

Learn • Practice • Grow

TEAM DETAILS

Team ID : SWTID-2026-2472
Team Size : 4
Team Leader : Lokeshwaran M
Team Member : Mohamed Noufal S
Team Member : Rakesh Sharma R
Team Member : Kamesh R

CERTIFICATE

This is to certify that the project titled “EduGenie: Google Gemini Powered Learning Assistant” has been documented as an academic project under the Naan Mudhalvan programme.

The project work describes an AI-powered learning assistant designed to support students through question answering, simplified explanations, quiz generation, summarization, and personalized learning recommendations.

Submitted by: Tella Divya Sree

Mentor: Siri

Date: 11/04/2025

Project Guide Signature: _____

Head of Department / Authorized Signatory: _____

DECLARATION

I hereby declare that this project documentation titled “EduGenie: Google Gemini Powered Learning Assistant” is prepared for academic submission under the Naan Mudhalvan project context.

The report is based on the supplied project documentation and presents the architecture, technologies, modules, workflow, testing activities, deployment approach, limitations, and future enhancements described for EduGenie.

I understand that the project uses generative AI and that generated responses should be reviewed for correctness and suitability before being relied upon for academic purposes.

Student Signature: _____

Name: Tella Divya Sree

Date: 11/04/2025

ACKNOWLEDGEMENT

I express my sincere gratitude to the mentor, faculty members, and the Naan Mudhalvan programme for providing an opportunity to explore a practical Generative AI application through the EduGenie project.

I am thankful for the guidance that supported the study of FastAPI, web development, Google Gemini integration, lightweight language models, prompt engineering, and user-oriented educational design.

I also acknowledge the value of the supplied project documentation, which provided the core architecture, functional modules, workflow, deployment steps, challenges, and future directions used as the foundation for this expanded report.

Finally, I thank everyone who contributed guidance and encouragement during the development and documentation of EduGenie.

ABSTRACT

EduGenie is a lightweight AI-powered educational assistant designed to simplify learning through generative artificial intelligence. The application supports students and self-learners by providing smart answers, simplified explanations, quizzes, summaries, and personalized learning recommendations.

The documented architecture uses FastAPI as the backend and a simple HTML and CSS interface rendered with Jinja2. Google Gemini 1.5 Pro is described as the cloud model for question answering, summarization, quiz generation, and learning-path recommendations, while LaMini-Flan-T5-783M is used locally for concept explanation.

The project follows a modular design in which each learning capability is separated into its own Python module and exposed through FastAPI endpoints. This makes the application easier to understand, test, maintain, and extend.

The project also demonstrates practical Generative AI concepts such as prompt construction, structured output, JSON parsing, error handling, cloud API integration, lightweight local inference, and responsive frontend integration.

The long-term vision described in the source includes voice interaction, multilingual support, mobile access, progress tracking, gamification, adaptive learning, collaboration features, LMS integration, and image/PDF-based learning assistance.

KEYWORDS AND PROJECT PROFILE

Keywords: Generative AI, Google Gemini, FastAPI, Learning Assistant, Question Answering, Quiz Generation, Summarization, Personalized Learning, LaMini-Flan-T5, Prompt Engineering.

Project Type: AI-powered educational web application.

Primary Users: Students, self-learners, and educational users.

Primary Interface: Web browser using HTML, CSS, and Jinja2.

Backend Framework: FastAPI.

Cloud AI: Google Gemini 1.5 Pro through API integration.

Local AI: LaMini-Flan-T5-783M for concept explanation.

Server: Uvicorn ASGI server.

The project emphasizes accessibility, simplicity, low-footprint operation, modularity, and future scalability.

TABLE OF CONTENTS

1. Introduction and Project Background
2. Project Objectives and Scope
3. Literature and Conceptual Background
4. System Requirement Analysis
5. System Architecture
6. Technology Stack
7. Prerequisites and Environment Setup
8. Project Workflow and Milestones
9. Model Selection and AI Strategy
10. Folder Architecture and Code Organization
11. Explanation Module
12. Question Answering Module
13. Quiz Generation Module
14. Summary Module
15. Learning Path Module
16. FastAPI Backend and API Design
17. Frontend Design and Integration
18. Implementation and Configuration
19. Testing and Validation
20. Results and Functional Review
21. Challenges, Limitations and Security
22. Future Enhancements and Deployment
23. Conclusion
24. References
25. Appendices and Project Submission Material

LIST OF FIGURES AND TABLES

Figure 1 – EduGenie System Architecture

Figure 2 – EduGenie Request Processing Workflow

Figure 3 – Functional Module Flow

Figure 4 – Project Folder Architecture

Figure 5 – Explanation Module Logic

Figure 6 – Q&A Module Logic

Figure 7 – Quiz Module Logic

Figure 8 – Summary Module Logic

Figure 9 – Learning Path Module Logic

Figure 10 – Frontend Integration

Table 1 – Functional Requirements

Table 2 – Non-Functional Requirements

Table 3 – Technology Stack

Table 4 – API Endpoints

Table 5 – Functional Test Cases

Table 6 – Risks and Mitigation Considerations

Table 7 – Future Enhancement Areas

1. INTRODUCTION

EduGenie is presented in the source document as a lightweight AI-powered educational assistant that simplifies learning through generative AI. The application is designed for students across academic levels and focuses on making educational interaction more direct, readable, and personalized.

The project combines a web interface with a FastAPI backend and AI model services. Instead of forcing learners to navigate separate tools for questions, explanations, practice, summaries, and learning plans, EduGenie brings these activities into a single learning-oriented interface.

The source describes three representative scenarios. A learner can ask a general question such as which ocean is the largest, generate a quiz to check understanding of the Pythagoras theorem, or request a structured learning path while exploring SQL. These examples illustrate the intended progression from information seeking to practice and guided learning.

The project is therefore not limited to a single chatbot function. It is organized as a set of educational capabilities that share a common interface and backend.

1.1 PROJECT BACKGROUND

Traditional digital learning resources often separate explanation, practice, revision, and learning planning. EduGenie attempts to connect these activities using generative AI. The source emphasizes clarity and accessibility, especially for beginners and learners who may find traditional educational content overwhelming.

A key design decision is the use of different models for different tasks. Gemini 1.5 Pro is used through an API for tasks that benefit from cloud-based generative capabilities, while LaMini-Flan-T5-783M is described as a lightweight local model for simplified concept explanation.

This division creates a practical hybrid architecture. Cloud intelligence supports several advanced generative tasks, while the local model provides a CPU-compatible option for explanations. The source notes that the application can work well on devices such as a Mac M1, emphasizing local efficiency together with cloud power.

The project also demonstrates how a modular application can be developed incrementally: select models, implement individual modules, expose them through APIs, connect a frontend, test the features, and run the application locally.

1.2 PROBLEM STATEMENT

Learners frequently need different types of academic assistance depending on the stage of learning. A learner may first need a direct answer, then a simpler explanation, followed by a short summary or quiz, and finally a structured path for deeper study.

The source project addresses this requirement by combining five capabilities: Q&A, explanation, quiz generation, summarization, and learning recommendations.

The problem is therefore framed as an educational interaction problem rather than only a question-answering problem. The assistant should make complex information easier to understand, convert learning material into practice questions, reduce long passages into useful revision material, and organize further study into progressive steps.

The system must also manage practical AI concerns such as response accuracy, output formatting, API errors, prompt quality, performance, and privacy.

2. PROJECT OBJECTIVES

The primary objective is to develop an accessible AI-powered learning assistant that supports multiple common study activities.

Specific objectives are:

2.1 DETAILED OBJECTIVES

1. Provide smart and concise answers to academic and general knowledge questions.
2. Explain difficult concepts in simple and readable language.
3. Generate three multiple-choice questions from a supplied passage or topic, with four options per question.
4. Summarize long educational passages while retaining core information.
5. Produce structured learning recommendations from beginner to advanced level.
6. Integrate AI capabilities into a lightweight FastAPI web application.
7. Provide a responsive HTML/CSS user interface with a simple task selection mechanism.
8. Demonstrate practical use of cloud AI and a lightweight local model in one modular application.
9. Support future expansion into voice, multilingual, mobile, analytics, gamification, and LMS integration.

2.2 PROJECT SCOPE

The documented scope covers a browser-based learning assistant with five main tasks. The user selects a task, supplies a question, topic, or passage, and receives the generated result in the interface.

The scope includes frontend forms, FastAPI endpoints, AI model calls, response handling, structured quiz output, and local execution through Uvicorn.

The source also identifies future scope rather than claiming these capabilities are already implemented. Future areas include voice interaction, multilingual support, mobile applications, progress dashboards, gamification, adaptive learning, group study, teacher and parent dashboards, LMS integration, image/PDF input, real-time synchronization, and smart notifications.

This distinction is important: the current project focuses on the five documented learning functions, while the additional items form an enhancement roadmap.

2.3 DEVELOPMENT METHODOLOGY

The project follows a milestone-oriented development approach documented in the source.

Milestone 1 focuses on model selection and architecture. Gemini 1.5 Pro is selected for Q&A, summarization, quizzes, and learning paths; LaMini-Flan-T5-783M is selected for concept explanation.

Milestone 2 focuses on core functionality development. Individual Python modules are created for explanation, Q&A, quiz generation, summarization, and learning recommendations. FastAPI endpoints are then connected to those modules.

Milestone 3 focuses on frontend development and live integration. The HTML interface contains a task dropdown, text area, and submit button, while CSS provides responsive styling.

Milestone 4 focuses on local execution and functional testing. The application is launched using Uvicorn and tested by asking questions, requesting explanations, generating quizzes, summarizing content, and requesting a learning plan.

3. SYSTEM REQUIREMENT ANALYSIS

The requirements are derived from the functions and setup described in the supplied project documentation.

Functional requirements describe what the system should do for the learner. Non-functional requirements describe qualities such as simplicity, responsiveness, maintainability, and reliable handling of AI/API responses.

The system also depends on a Google Gemini API key for the cloud-powered modules and Python 3.10+ for the application environment.

3.1 FUNCTIONAL REQUIREMENTS

Table 1 summarizes the major functional requirements.

ID	Requirement	Description
FR-01	Question answering	Answer academic/general questions through /qa
FR-02	Concept explanation	Provide simplified explanation through /explain
FR-03	Quiz generation	Generate 3 MCQs with 4 options each
FR-04	Summarization	Condense long passages through /summarize
FR-05	Learning path	Generate structured recommendations through /learn/recommendations
FR-06	Frontend interaction	Allow task selection, text input and submission

3.2 NON-FUNCTIONAL REQUIREMENTS

The project documentation emphasizes a lightweight, accessible interface and cloud/local model combination. The following quality requirements reflect the documented design goals.

- Usability: task selection and text input should be straightforward.
- Responsiveness: the frontend should display results after backend processing.
- Maintainability: each AI capability is kept in a separate module.
- Reliability: API and output parsing errors should be handled with useful messages.
- Compatibility: the documented setup uses Python 3.10+ and standard web technologies.
- Performance awareness: the local explanation model is selected partly for CPU compatibility, while cloud inference is used where appropriate.
- Security awareness: API credentials should be stored securely rather than exposed in frontend code.

Quality	**Expectation**
Usability	Simple task-based web interface
Maintainability	Separate Python modules for each function
Reliability	Handle AI/API and parsing errors
Compatibility	Python 3.10+ and browser-based access
Security	Secure storage of API credentials
Performance	Lightweight local model plus cloud inference

3.3 FEASIBILITY ANALYSIS

Technical feasibility is supported by the use of widely used Python web technologies and accessible AI APIs. The source specifies FastAPI, Uvicorn, Jinja2, HTML/CSS, Gemini API access, and a lightweight local model.

Operational feasibility follows from the simple interaction model: choose a task, enter content, submit, and view the response.

Development feasibility is supported by modular separation. Each feature can be implemented and tested independently before integration.

Scalability is identified as a future advantage of the cloud-based backend. However, the supplied documentation does not provide production load measurements, so scalability is treated as an architectural direction rather than a measured performance result.

4. SYSTEM ARCHITECTURE

EduGenie uses a layered architecture. At the top is the learner using a web browser. The frontend provides the task selection and input interface. FastAPI acts as the backend application layer and routes requests to the relevant learning module.

The module layer contains five capabilities: explanation, Q&A, quiz, summary, and learning path. The explanation module uses LaMini-Flan-T5-783M according to the source, while the other four are powered by Gemini 1.5 Pro.

The modular architecture reduces coupling. A change to quiz prompting, for example, can remain within the quiz module without requiring changes to the summary or explanation logic.

4.1 ARCHITECTURE DIAGRAM

The following diagram represents the documented architecture and the relationship between the browser, frontend, FastAPI backend, individual learning modules, and AI services.

Figure: EduGenie System Architecture

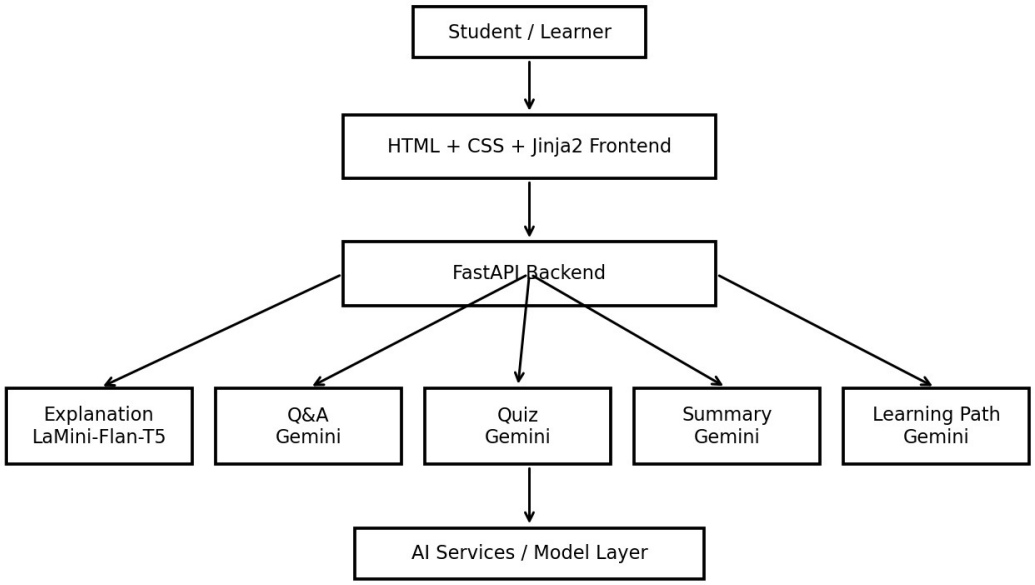


Figure 1: EduGenie System Architecture

5. TECHNOLOGY STACK

The technology choices in the source are focused on simplicity and practical AI integration.

Technology	Purpose	Role
Python 3.10+	Programming language	Backend/model integration
FastAPI	Web framework	REST endpoints
Uvicorn	ASGI server	Run FastAPI locally
HTML/CSS	Frontend	User interface
Jinja2	Template engine	Render index.html
Gemini 1.5 Pro	Cloud AI	Q&A, quiz, summary, learning path
LaMini-Flan-T5-783M	Local AI	Concept explanation

5.1 TECHNOLOGY JUSTIFICATION

Python 3.10+ provides the main programming environment. FastAPI is used to create the backend and REST-style endpoints. Uvicorn serves the FastAPI application through ASGI.

HTML and CSS provide the frontend structure and presentation. Jinja2 is used as the HTML templating engine for the templates directory.

Google Gemini is used for generative tasks requiring cloud inference. LaMini-Flan-T5-783M is selected for local concept explanation because the source describes it as instruction-tuned, lightweight, and CPU-compatible.

Together, these technologies create a relatively small project footprint while still demonstrating multiple Generative AI integration patterns.

Layer	**Technology**	**Reason in project**
Language	Python 3.10+	Main development environment
Backend	FastAPI	RESTful application endpoints
Frontend	HTML/CSS	Simple responsive interface
Templates	Jinja2	HTML templating
AI	Gemini + LaMini-Flan-T5	Cloud and local generative capabilities
Server	Uvicorn	ASGI local execution

6. PREREQUISITES

The source lists Python 3.10+, FastAPI, HTML/CSS, a Google Gemini API key, Uvicorn, and Jinja2 as prerequisites.

Python installation requires downloading Python 3.10+ and ensuring that Python is available on the system PATH. The source recommends verifying the installation with python --version.

A Google AI Studio account is required to create and securely store the Gemini API key.

Uvicorn and Jinja2 can be installed through pip as documented in the source.

```
python --version
pip install uvicorn
pip install jinja2
```

6.1 ENVIRONMENT SETUP

A basic local setup sequence is:

1. Install Python 3.10+.
2. Open the EduGenie project folder in VS Code.
3. Create or activate a Python virtual environment.
4. Install dependencies listed in requirements.txt.
5. Configure the Gemini API key in the environment.
6. Start the FastAPI application with Uvicorn.
7. Open http://127.0.0.1:8000 in a browser.

The source specifically documents the Uvicorn command: uvicorn main:app --reload.

7. PROJECT WORKFLOW AND MILESTONES

The project workflow is organized into four milestones: model selection and architecture; core functionality development; frontend development and integration; and deployment/testing.

This sequence reduces development risk because the AI strategy and application structure are established before the user interface is connected.

The workflow also reflects an iterative engineering pattern: define the capability, implement its module, expose it through an endpoint, integrate the frontend, and test the complete path.

7.1 PROJECT WORKFLOW DIAGRAM

The following diagram summarizes the request-processing flow from user input through the selected module and AI model to the final frontend response.

Figure: EduGenie Request Processing Workflow



Figure 2: EduGenie Request Processing Workflow

8. MODEL SELECTION AND AI STRATEGY

The source explicitly identifies two AI models and assigns them different responsibilities.

Gemini 1.5 Pro is used via API for Q&A, summarization, quiz generation, and learning paths. The stated benefits are advanced reasoning, structured outputs, and cloud inference.

LaMini-Flan-T5-783M is used locally for concept explanation. The stated benefits are instruction tuning, lightweight operation, and CPU compatibility.

This division illustrates task-oriented model selection. Rather than using one model for every capability, the project matches model characteristics to the educational task.

Model	Used For	Benefits stated in source
Gemini 1.5 Pro	Q&A, summarization, quiz, learning paths	Advanced reasoning; structured outputs; cloud inference
LaMini-Flan-T5-783M	Concept explanation	Instruction-tuned; lightweight; CPU-compatible

9. FOLDER ARCHITECTURE

The source specifies the following project structure:

```
EduGenie/
├── main.py
├── explanation_module.py
├── qna.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── templates/
│   └── index.html
├── static/
│   └── style.css
└── requirements.txt
```

9.1 CODE ORGANIZATION

main.py acts as the FastAPI application entry point. The remaining Python files separate educational functions into modules. The templates directory contains index.html, while static contains style.css. requirements.txt records Python dependencies.

This organization supports modular development and makes it easier to identify where each feature is implemented.

Figure: Functional Module Flow

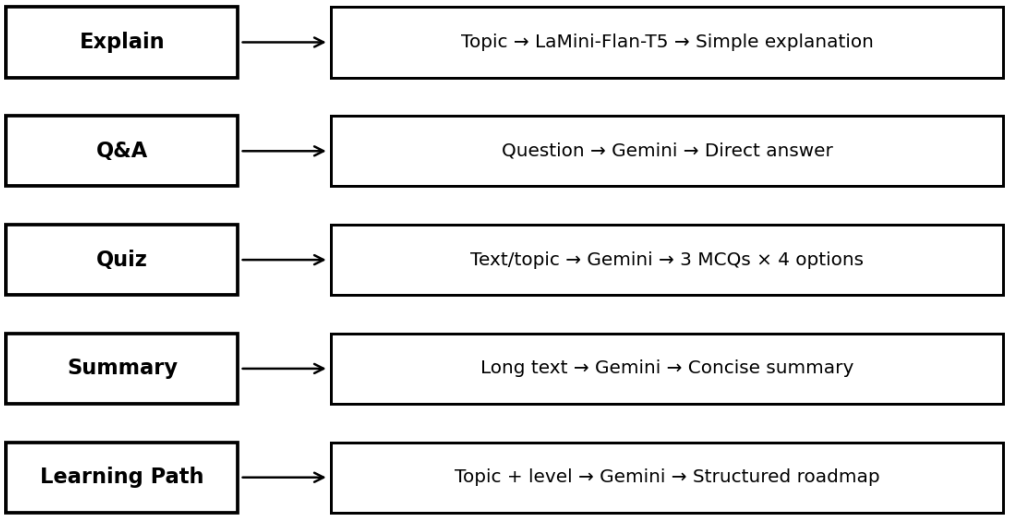


Figure 3: Functional Module Flow

10. EXPLANATION MODULE

The Explanation Module is designed to make complex topics understandable to learners with limited background knowledge. The source uses LaMini-Flan-T5-783M for this function.

The module creates an instruction prompt that asks for a simple and easy-to-understand explanation. The model then generates the educational response.

The design emphasizes clarity and brevity. This is particularly relevant for beginners, school students, and self-learners who may find technical descriptions difficult to follow.

The module also demonstrates local inference as part of the hybrid AI architecture.

10.1 EXPLANATION MODULE LOGIC

Concept explanation flow:

1. Receive a topic from the user.
2. Build an instructional prompt requesting a simple explanation.
3. Pass the prompt to the local LaMini-Flan-T5 model.
4. Generate the response.
5. Return the generated explanation to the FastAPI endpoint.
6. Display the result in the web interface.

The source describes the model as context-aware and suitable for concise educational responses.

Figure: Functional Module Flow

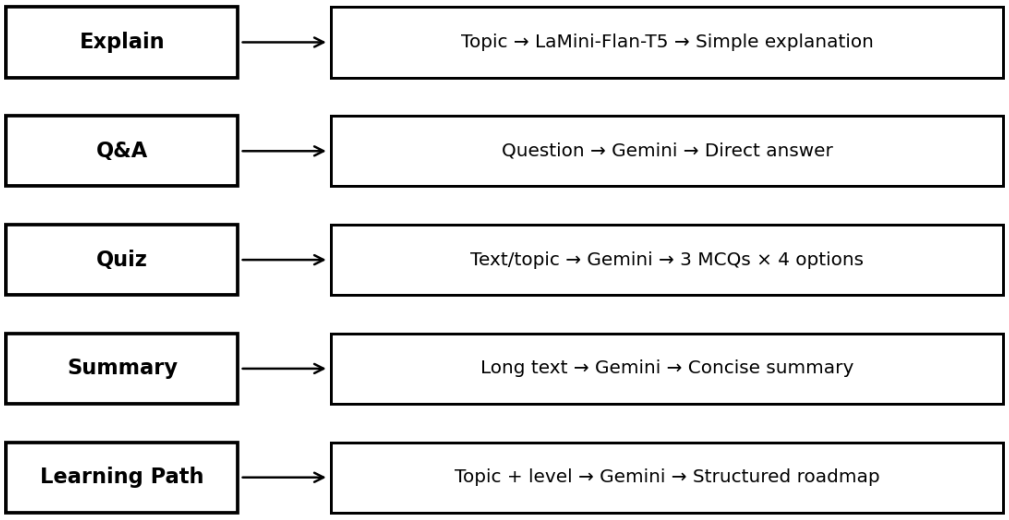


Figure 5: Functional Module Flow (including explanation task)

11. QUESTION ANSWERING MODULE

The Q&A module uses Gemini 1.5 Pro. It is intended for general knowledge and academic questions and uses the model's contextual capabilities to produce answers.

A typical interaction begins when the learner enters a question. The frontend sends the request to the backend, the /qa endpoint invokes the Q&A module, and the module submits the question to Gemini.

The generated answer is returned through the API and displayed to the learner. The architecture keeps the web layer independent of the AI implementation.

Step	Action
1	Receive learner question
2	Send request to /qa
3	Invoke Q&A module
4	Generate response using Gemini
5	Return response to frontend

12. QUIZ GENERATION MODULE

The Quiz Module converts a passage or topic into a short assessment. The source specifies three multiple-choice questions, each containing four options.

The module asks Gemini to understand the supplied content, create relevant questions, and generate plausible distractors. The expected result is structured JSON.

The source also describes a clean_json_block function used to remove Markdown code fences before JSON parsing. This is an important integration step because a model may return structured data inside a Markdown code block.

If generation or parsing fails, the module returns a detailed error message for debugging.

```
Expected conceptual structure:
[
  {
    "question": "...",
    "options": ["A", "B", "C", "D"],
    "correct_answer": "..."
  }
]
```

12.1 QUIZ VALIDATION AND USER INTERACTION

The quiz workflow supports a learning-by-practice interaction. The learner receives questions and options, selects answers, and the application can indicate when an answer is incorrect according to the project description.

The documented design focuses on relevance and plausible distractors. The structured format also makes the generated quiz easier for the frontend to process.

A robust implementation should validate that each question has four options and that a correct answer is present before displaying the quiz. The supplied source establishes the JSON parsing approach but does not specify a complete validation schema.

13. SUMMARY MODULE

The Summary Module uses Gemini to convert long educational passages into concise and easy-to-understand versions.

The source states that the feature aims to retain core information while removing redundancy. This makes it useful for quick revision.

The summary flow is straightforward: accept long text, create a summarization prompt, send the request to Gemini, receive the generated summary, and display it to the user.

The module is especially useful when learners need to review large passages in less time while preserving the main ideas.

Input	Processing	Output
Long educational passage	Gemini summarization prompt	Concise revision-oriented summary

14. LEARNING PATH MODULE

The Learning Path Module generates a structured learning plan for a topic. The source describes the get_learning_recommendations function as using Gemini to suggest concepts from beginner to advanced levels.

The prompt instructs the model to organize concepts by difficulty and provide resources such as videos, articles, or books. The path is intended to adapt to the learner's level.

The output therefore serves a planning function rather than simply answering one question. It helps a learner understand what to study first, what to learn next, and how to progress.

Stage	**Typical purpose**
Beginner	Foundational concepts
Intermediate	Core skills and practice
Advanced	Deeper concepts and application
Resources	Videos, articles, or books as suggested by AI

14.1 LEARNING PATH OUTPUT DESIGN

A useful learning path should be sequential and understandable. The source specifically emphasizes beginner-to-advanced concepts, timelines, and study suggestions.

The recommended structure can therefore include: starting concepts, intermediate topics, advanced topics, suggested practice, and learning resources.

The source states that the function safely handles API responses and errors. If the response is invalid or incomplete, the application should provide a helpful error message rather than silently displaying unusable output.

15. FASTAPI BACKEND AND API DESIGN

FastAPI forms the central backend layer. The source lists five RESTful endpoints, each connected to a corresponding module.

Endpoint	Input	Purpose
/qa	Question	Gemini-powered answer
/explain	Topic	Simplified explanation
/quiz	Topic/passage	Structured quiz
/summarize	Long text	Concise summary
/learn/recommendations	Topic	Learning path

15.1 API ENDPOINTS

The documented endpoints are /qa, /explain, /quiz, /summarize, and /learn/recommendations.

Each endpoint receives input from the frontend and returns the generated result. Keeping separate routes makes the backend easy to test and maps directly to the five user-facing tasks.

20.2 LIMITATIONS

The supplied documentation does not provide measured accuracy percentages, response latency benchmarks, concurrent-user capacity, or a formal user study. Therefore, this report does not invent numerical performance results.

The current documented feature set is limited to five learning functions. Voice, multilingual interaction, mobile applications, progress tracking, gamification, collaboration, LMS integration, and image/PDF input are described as future enhancements rather than current capabilities.

Generative responses also require human review when used for important academic decisions.

22.1 FUTURE ENHANCEMENTS

The source proposes voice-based interaction, multilingual support, mobile access, progress tracking, gamification, adaptive learning paths, group study, teacher and parent dashboards, LMS integration, image/PDF input, real-time synchronization, and smart notifications.

These items are presented as future directions rather than current implemented functions.

23. CONCLUSION

EduGenie is documented as a robust and accessible AI-powered educational assistant that combines cloud-based intelligence with a lightweight web interface.

The project brings together question answering, simplified explanation, quiz generation, summarization, and personalized learning paths. Its modular architecture separates these capabilities while maintaining a common FastAPI backend and web frontend.

The project provides practical exposure to prompt engineering, cloud AI integration, local model inference, structured outputs, web APIs, frontend integration, and user-centered design.

The source concludes that EduGenie can become a broader learning companion through future additions such as voice, multilingual support, mobile access, analytics, adaptive learning, collaboration, and multimodal input.

24. REFERENCES

1. Google AI for Developers – documentation and guidance for Google AI/Gemini API usage, as referenced in the supplied project document.

2. FastAPI Documentation – framework documentation referenced in the supplied project document.

3. Jinja2 Documentation – templating documentation referenced in the supplied project document.

4. PyPI / Uvicorn – package and installation information referenced in the supplied project document.

5. GeeksforGeeks – Gemini API key setup guide referenced in the supplied project document.

6. Tom’s Hardware – Python installation guidance referenced in the supplied project document.

7. Python Programming Tutorial – Full Course for Beginners, referenced as a learning resource in the supplied project document.

8. Supplied project document: “EduGenie: Google Gemini Powered Learning Assistant”, 18-page source documentation, submitted by Tella Divya Sree, mentor Siri, dated 11/04/2025.

Note: The references are limited to resources explicitly named or described in the supplied project document; no additional external research has been inserted.

25. APPENDIX A – PROJECT FILES

The source defines the following folder architecture:

```
EduGenie/
  main.py
  explanation_module.py
  qna.py
  quiz_module.py
  summary_module.py
  learning_path.py
  templates/
    index.html
  static/
    style.css
  requirements.txt
```


```
EduGenie/
  main.py
  explanation_module.py
  qna.py
  quiz_module.py
  summary_module.py
  learning_path.py
  templates/
    index.html
  static/
    style.css
  requirements.txt
```

APPENDIX B – SAMPLE CONFIGURATION

The supplied project requires a Google Gemini API key. The source specifies that the key should be generated through Google AI Studio and stored securely.

A configuration pattern should keep secrets outside source-controlled files. The following is a documentation example rather than a claim that the original source contained this exact file.

```
# Example configuration pattern
GEMINI_API_KEY=your_api_key_here

# Keep the real key private.
# Do not place secrets in HTML, CSS, or client-side JavaScript.
```


```
# Example configuration pattern
GEMINI_API_KEY=your_api_key_here

# Keep the real key private.
# Do not place secrets in HTML, CSS, or client-side JavaScript.
```

APPENDIX C – SAMPLE API REFERENCE

The source identifies the following endpoints and their purposes:

Method	Endpoint	Purpose
POST	/qa	Question answering
POST	/explain	Concept explanation
POST	/quiz	Quiz generation
POST	/summarize	Text summarization
POST	/learn/recommendations	Learning recommendations

Method	Endpoint	Purpose
POST	/qa	Question answering
POST	/explain	Concept explanation
POST	/quiz	Quiz generation
POST	/summarize	Text summarization
POST	/learn/recommendations	Learning recommendations

APPENDIX D – DEPLOYMENT CHECKLIST

1. Install Python 3.10+.
2. Confirm Python is available through the command line.
3. Install the project dependencies.
4. Configure the Gemini API key securely.
5. Confirm templates and static files are present.
6. Start Uvicorn using uvicorn main:app --reload.
7. Open http://127.0.0.1:8000.
8. Test all five documented functions.

```
uvicorn main:app --reload

Open: http://127.0.0.1:8000
```

APPENDIX E – PROJECT DETAILS AND UNDERTAKING

Project Title: EduGenie: Google Gemini Powered Learning Assistant

Project Category: Generative AI / Educational Technology

Programme Context: Naan Mudhalvan

Submitted by: Tella Divya Sree

Mentor: Siri

Source Document Date: 11/04/2025

Backend: FastAPI

Frontend: HTML + CSS + Jinja2

Cloud AI: Gemini 1.5 Pro

Local AI: LaMini-Flan-T5-783M

Student undertaking: I confirm that I have reviewed the project documentation and understand the main technologies, architecture, functional modules, and workflow of EduGenie. I understand that generative AI responses can require verification and that the Gemini API key must be handled securely.

Student Signature: _____ Date: _____

APPENDIX F – FUNCTIONAL TEST RECORD

The project documentation describes functional testing for five activities: asking a question, getting an explanation, generating a quiz, summarizing content, and receiving a personalized learning plan.

Test record template:

TC-01 – Q&A: Ask a factual or academic question. Expected: relevant answer displayed.

TC-02 – Explanation: Enter a complex topic. Expected: simple explanation displayed.

TC-03 – Quiz: Enter a topic or passage. Expected: three questions with four options each.

TC-04 – Summary: Paste a long educational passage. Expected: concise summary displayed.

TC-05 – Learning Path: Enter a topic such as SQL. Expected: structured beginner-to-advanced guidance.

The exact generated responses are not reproduced because the supplied project document does not provide their full text.

TEAM DETAILS

Team ID : SWTID-2026-2472
Team Size : 4
Team Leader : Lokeshwaran M
Team Member : Mohamed Noufal S
Team Member : Rakesh Sharma R
Team Member : Kamesh R